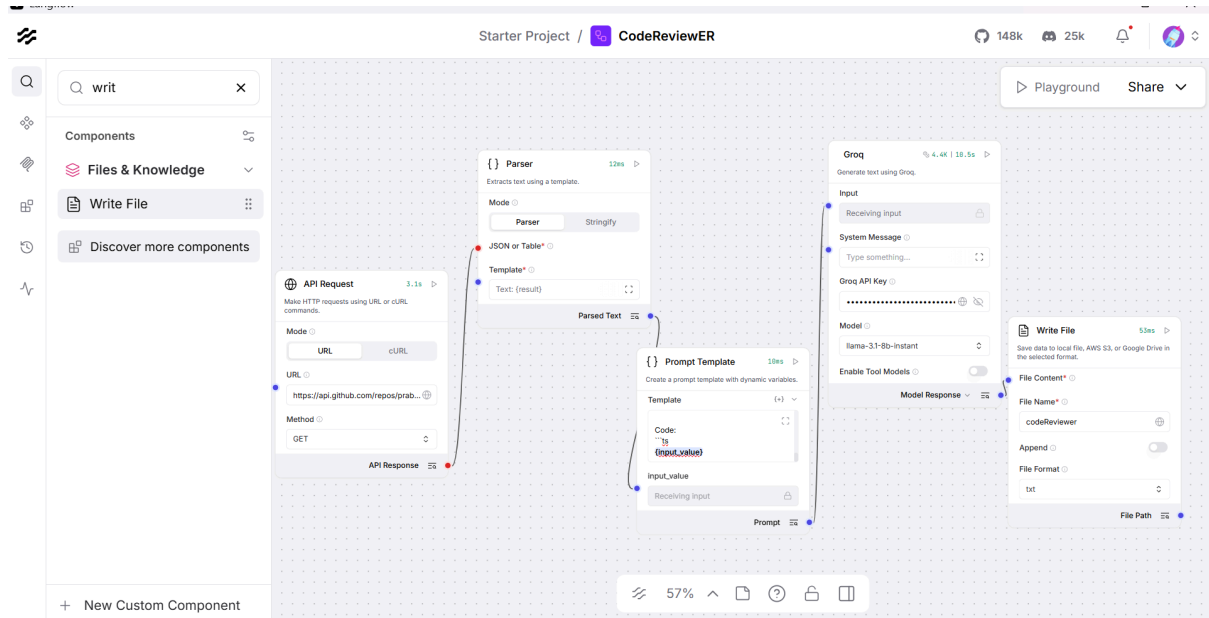


Code Reviewer

Lang flow:



Prompt:

Instruction

You are required to review and validate the provided TypeScript (.ts) code.

Analyze the code for:

- correctness
- type safety
- readability
- maintainability
- performance
- adherence to best practices

Identify issues, suggest improvements, and provide a refined version of the code.

Context

The input TypeScript code may contain:

- Type errors or weak typing (any, missing interfaces)
- Logical or runtime issues
- Poor naming conventions
- Lack of modularity or reusability
- Missing error handling

- Violations of TypeScript or general coding best practices

Persona

You are a Senior TypeScript Developer with 10+ years of experience in building scalable applications.

You provide precise, actionable, and practical feedback.

Tone

- Professional
- Concise
- Constructive
- Technically precise

Output Format

Issues:

- List all problems clearly

Suggestions:

- Provide actionable improvements

Improved Code:

- Provide a corrected and optimized version

Code:

```
``ts  
{input_value}
```

Original Code:

```
import express from 'express'  
import fetch from 'node-fetch'  
import https from 'https'  
import { RadarGenerateSchema, RadarGenerateResponseSchema,  
TestableRequirementRequestSchema, TestableRequirementResponseSchema } from  
'../schemas'  
import { buildRadarPrompt, buildRadarSystemPrompt, buildTestableRequirementPrompt,  
buildTestableRequirementSystemPrompt } from '../prompt'
```

```

import { callKiroCli, validateKiroCliAuth } from '../llm/kiroCliClient'

export const radarRouter = express.Router()

const httpsAgent = new https.Agent({ rejectUnauthorized: false })

function extractFromHtml(html: string, cardId: string): { cardId: string; subject: string;
description: string } | null {
  let subject = ""
  let description = ""

  const titleMatch = html.match(/<title[^>]*#[^#]*#\d+[:\s]+([^\<-]+)/i)
  if (titleMatch) subject = titleMatch[1].trim()

  if (!subject) {
    const subjectDiv = html.match(/<div[^>]+class="[^"]*"subject["'"]*["^>"]*>([\s\S]*?)<\div>/i)
    if (subjectDiv) subject = subjectDiv[1].replace(/<[^>]+>/g, "").trim()
  }
  if (!subject) {
    const h2 = html.match(/<h2[^>]*>([\s\S]*?)<\h2>/i)
    if (h2) subject = h2[1].replace(/<[^>]+>/g, "").trim()
  }

  const wikiMatch =
html.match(/<div[^>]+(?:class="[^"]*"wiki["'"]*"id="["']*"description["'"]*)"["^>"]*>([\s\S]*?)<\div>/i)
  if (wikiMatch) {
    description = wikiMatch[1].replace(/<[^>]+>/g, ' ').replace(/\s+/g, ' ').trim()
  }

  if (!subject) return null
  return { cardId, subject, description }
}

radarRouter.get('/auth/validate', async (_req: express.Request, res: express.Response):
Promise<void> => {
  const isValid = await validateKiroCliAuth()
  if (isValid) {
    res.json({ authenticated: true })
  } else {
    res.status(401).json({ authenticated: false, error: 'Kiro CLI not authenticated. Run kiro-cli
login first.' })
  }
})

radarRouter.get('/card/:cardId', async (req: express.Request, res: express.Response):
Promise<void> => {
  const { cardId } = req.params
  const authToken = (req.headers['x-radar-token'] as string || "").trim()

```

```

const radarUrl = (req.headers['x-radar-url'] as string || "").trim()

if (!authToken || !radarUrl) {
  res.status(400).json({ error: 'Missing x-radar-token or x-radar-url header' })
  return
}

const baseUrl = radarUrl.replace(/\/$/, "")
const url = `${baseUrl}/issues/${cardId}`

const radarHeaders: Record<string, string> = {
  'Accept': '*/*',
  'X-Redmine-API-Key': authToken,
}

try {
  const response = await fetch(url, { headers: radarHeaders, agent: httpsAgent })
  const bodyText = await response.text()

  if (response.status === 401) {
    res.status(401).json({ error: 'Invalid Radar API key.' })
    return
  }
  if (response.status === 403) {
    res.status(403).json({ error: `Access denied to card #${cardId}.` })
    return
  }
  if (response.status === 404) {
    res.status(404).json({ error: `Card #${cardId} not found.` })
    return
  }
  if (!response.ok) {
    res.status(response.status).json({ error: `Radar returned HTTP ${response.status}` })
    return
  }

  if (bodyText.trim().startsWith('{')) {
    try {
      const d = JSON.parse(bodyText)
      const iss = d?.issue
      if (iss) {
        res.json({ cardId, subject: iss.subject || "", description: iss.description || "" })
        return
      }
    } catch { /* fall through to HTML */ }
  }

  if (bodyText.includes('<html') || bodyText.includes('<title')) {

```

```

    const parsed = extractFromHtml(bodyText, cardId)
    if (parsed) {
      res.json(parsed)
      return
    }
  }

  res.status(404).json({ error: `Could not extract data from card #${cardId}.` })
} catch (err) {
  res.status(502).json({ error: `Network error: ${err as Error}.message` })
}
})

```

```

radarRouter.post('/testable-requirement', async (req: express.Request, res:
express.Response): Promise<void> => {
  const validation = TestableRequirementRequestSchema.safeParse(req.body)
  if (!validation.success) {
    res.status(400).json({ error: validation.error.message })
    return
  }
}

```

```

const { description } = validation.data
const systemPrompt = buildTestableRequirementSystemPrompt()
const userPrompt = buildTestableRequirementPrompt(description)
const fullPrompt = `${systemPrompt}\n\n${userPrompt}`

```

```

try {
  const output = await callKiroCli(fullPrompt)

  let parsed: any
  try {
    parsed = JSON.parse(output)
  } catch {
    const jsonMatch = output.match(/\{[\s\S]*\}/)
    if (jsonMatch) {
      try {
        parsed = JSON.parse(jsonMatch[0])
      } catch {
        res.status(502).json({ error: 'Kiro CLI returned invalid JSON', raw: output })
        return
      }
    } else {
      res.status(502).json({ error: 'Kiro CLI returned invalid JSON', raw: output })
      return
    }
  }
}

```

```

const responseValidation = TestableRequirementResponseSchema.safeParse(parsed)

```

```

    if (!responseValidation.success) {
      res.status(502).json({ error: 'Kiro CLI response schema mismatch', raw: output })
      return
    }

    res.json(responseValidation.data)
  } catch (err) {
    res.status(502).json({ error: (err as Error).message })
  }
})

radarRouter.post('/generate', async (req: express.Request, res: express.Response):
Promise<void> => {
  const validation = RadarGenerateSchema.safeParse(req.body)
  if (!validation.success) {
    res.status(400).json({ error: validation.error.message })
    return
  }

  const { cardId, subject, description, testableRequirement, testTypes } = validation.data
  const systemPrompt = buildRadarSystemPrompt(testTypes)
  const userPrompt = buildRadarPrompt(cardId, subject, description, testableRequirement,
testTypes)
  const fullPrompt = `${systemPrompt}\n\n${userPrompt}`

  try {
    const output = await callKiroCli(fullPrompt)

    let parsed: any
    try {
      parsed = JSON.parse(output)
    } catch {
      const jsonMatch = output.match(/\{[^\s\S]*\}/)
      if (jsonMatch) {
        try {
          parsed = JSON.parse(jsonMatch[0])
        } catch {
          res.status(502).json({ error: 'Kiro CLI returned invalid JSON', raw: output.substring(0,
500) })
          return
        }
      } else {
        res.status(502).json({ error: 'Kiro CLI returned invalid JSON', raw: output.substring(0,
500) })
        return
      }
    }
  }
}

```

```

const responseValidation = RadarGenerateResponseSchema.safeParse({
  cases: parsed.cases,
  cardId,
  subject,
  model: 'kiro-cli',
  promptTokens: 0,
  completionTokens: 0
})

if (!responseValidation.success) {
  res.status(502).json({ error: 'Kiro CLI response schema mismatch' })
  return
}

res.json(responseValidation.data)
} catch (err) {
  res.status(502).json({ error: (err as Error).message })
}
})

```

Lang Output:

****Issues:****

1. ****Type Errors:****

- The `https` module is not imported correctly. It should be `https` instead of `https`.
- The `https.Agent` is created with `rejectUnauthorized` set to `false`, which is a security risk. It's better to handle certificate verification explicitly.
- The `fetch` function is used without specifying the `mode` option, which can lead to issues with CORS.
- The `res.json()` method is called without checking if the response is already sent.

2. ****Logical Issues:****

- The `extractFromHtml` function does not handle cases where the HTML is malformed or missing required tags.
- The `radarRouter.post('/testable-requirement')` and `radarRouter.post('/generate')` handlers do not check if the `req.body` is an object before calling `safeParse` on it.
- The `callKiroCli` function is called without checking if the `fullPrompt` is a string.

3. ****Readability and Maintainability:****

- The code is not modular and has a lot of repeated logic. It would be better to break it down into smaller functions or modules.
- The variable names are not descriptive and do not follow a consistent naming convention.

- The code uses a lot of global variables and functions, which makes it harder to understand and maintain.

4. ****Performance:****

- The code uses a lot of synchronous operations, which can block the event loop and lead to performance issues.
- The `fetch` function is used without specifying the `timeout` option, which can lead to issues with slow network connections.

5. ****Best Practices:****

- The code does not follow the principle of least privilege and uses a lot of global variables and functions.
- The code does not handle errors and exceptions properly.
- The code does not follow the SOLID principles of object-oriented design.

****Suggestions:****

1. ****Use a more secure way to create the `https.Agent`**:**

- Instead of setting `rejectUnauthorized` to `false`, handle certificate verification explicitly using the `rejectUnauthorized` option.

2. ****Specify the `mode` option for the `fetch` function**:**

- Use the `mode` option to specify the CORS mode, which can help prevent issues with CORS.

3. ****Check if the response is already sent before calling `res.json()`**:**

- Use the `res.finished` property to check if the response is already sent before calling `res.json()`.

4. ****Break down the code into smaller functions or modules**:**

- Use functions or modules to encapsulate repeated logic and make the code more modular.

5. ****Use more descriptive variable names and follow a consistent naming convention**:**

- Use variable names that are descriptive and follow a consistent naming convention.

6. ****Use asynchronous operations instead of synchronous operations**:**

- Use asynchronous operations instead of synchronous operations to avoid blocking the event loop.

7. ****Handle errors and exceptions properly**:**

- Use try-catch blocks to handle errors and exceptions properly.

8. ****Follow the SOLID principles of object-oriented design**:**

- Use the SOLID principles to design the code in a more modular and maintainable way.

****Improved Code:****


```

``typescript
import express from 'express';
import fetch from 'node-fetch';
import https from 'https';
import { RadarGenerateSchema, RadarGenerateResponseSchema,
TestableRequirementRequestSchema, TestableRequirementResponseSchema } from
'../schemas';
import { buildRadarPrompt, buildRadarSystemPrompt, buildTestableRequirementPrompt,
buildTestableRequirementSystemPrompt } from '../prompt';
import { callKiroCli, validateKiroCliAuth } from '../llm/kiroCliClient';

const radarRouter = express.Router();

const httpsAgent = new https.Agent({
  rejectUnauthorized: true,
  ca: [process.env.RADAR_CA_CERT],
});

function extractFromHtml(html: string, cardId: string): { cardId: string; subject: string;
description: string } | null {
  // ...
}

radarRouter.get('/auth/validate', async (req: express.Request, res: express.Response):
Promise<void> => {
  const isValid = await validateKiroCliAuth();
  if (isValid) {
    res.json({ authenticated: true });
  } else {
    res.status(401).json({ authenticated: false, error: 'Kiro CLI not authenticated. Run kiro-cli
login first.' });
  }
});

radarRouter.get('/card/:cardId', async (req: express.Request, res: express.Response):
Promise<void> => {
  const { cardId } = req.params;
  const authToken = (req.headers['x-radar-token'] as string || "").trim();
  const radarUrl = (req.headers['x-radar-url'] as string || "").trim();

  if (!authToken || !radarUrl) {
    res.status(400).json({ error: 'Missing x-radar-token or x-radar-url header' });
    return;
  }

  const baseUrl = radarUrl.replace(/\/$/, "");
  const url = `${baseUrl}/issues/${cardId}`;

```

```

const radarHeaders: Record<string, string> = {
  'Accept': '*/*',
  'X-Redmine-API-Key': authToken,
};

try {
  const response = await fetch(url, {
    headers: radarHeaders,
    agent: httpsAgent,
    mode: 'cors',
    timeout: 5000,
  });

  const bodyText = await response.text();

  if (response.status === 401) {
    res.status(401).json({ error: 'Invalid Radar API key.' });
    return;
  }

  if (response.status === 403) {
    res.status(403).json({ error: `Access denied to card #${cardId}.` });
    return;
  }

  if (response.status === 404) {
    res.status(404).json({ error: `Card #${cardId} not found.` });
    return;
  }

  if (!response.ok) {
    res.status(response.status).json({ error: `Radar returned HTTP ${response.status}` });
    return;
  }

  if (bodyText.trim().startsWith('{')) {
    try {
      const d = JSON.parse(bodyText);
      const iss = d.issue;
      if (iss) {
        res.json({ cardId, subject: iss.subject || '', description: iss.description || '' });
        return;
      }
    } catch { /* fall through to HTML */ }
  }

  if (bodyText.includes('<html') || bodyText.includes('<title')) {
    const parsed = extractFromHtml(bodyText, cardId);
  }

```

```

    if (parsed) {
      res.json(parsed);
      return;
    }
  }

  res.status(404).json({ error: `Could not extract data from card #${cardId}.` });
} catch (err) {
  res.status(502).json({ error: `Network error: ${err as Error}.message` });
}
});

radarRouter.post('/testable-requirement', async (req: express.Request, res:
express.Response): Promise<void> => {
  const validation = TestableRequirementRequestSchema.safeParse(req.body);
  if (!validation.success) {
    res.status(400).json({ error: validation.error.message });
    return;
  }

  const { description } = validation.data;
  const systemPrompt = buildTestableRequirementSystemPrompt();
  const userPrompt = buildTestableRequirementPrompt(description);
  const fullPrompt = `${systemPrompt}\n\n${userPrompt}`;

  try {
    const output = await callKiroCli(fullPrompt);

    let parsed: any;
    try {
      parsed = JSON.parse(output);
    } catch {
      const jsonMatch = output.match(/(?:\s\S)*\s/);
      if (jsonMatch) {
        try {
          parsed = JSON.parse(jsonMatch[0]);
        } catch {
          res.status(502).json({ error: 'Kiro CLI returned invalid JSON', raw: output.substring(0,
500) });
          return;
        }
      } else {
        res.status(502).json({ error: 'Kiro CLI returned invalid JSON', raw: output.substring(0,
500) });
        return;
      }
    }
  }
}

```

```

const responseValidation = TestableRequirementResponseSchema.safeParse(parsed);
if (!responseValidation.success) {
  res.status(502).json({ error: 'Kiro CLI response schema mismatch', raw: output });
  return;
}

res.json(responseValidation.data);
} catch (err) {
  res.status(502).json({ error: (err as Error).message });
}
});

```

```

radarRouter.post('/generate', async (req: express.Request, res: express.Response):
Promise<void> => {

```

```

  const validation = RadarGenerateSchema.safeParse(req.body);
  if (!validation.success) {
    res.status(400).json({ error: validation.error.message });
    return;
  }

```

```

const { cardId, subject, description, testableRequirement, testTypes } = validation.data;
const systemPrompt = buildRadarSystemPrompt(testTypes);
const userPrompt = buildRadarPrompt(cardId, subject, description, testableRequirement,
testTypes);
const fullPrompt = `${systemPrompt}\n\n${userPrompt}`;

```

```

try {
  const output = await callKiroCli(fullPrompt);

  let parsed: any;
  try {
    parsed = JSON.parse(output);
  } catch {
    const jsonMatch = output.match(/([\s\S]*)/);
    if (jsonMatch) {
      try {
        parsed = JSON.parse(jsonMatch[0]);
      } catch {
        res.status(502).json({ error: 'Kiro CLI returned

```